E0 and E4: does the generator matter to index calculus?

PLAN §4, experiments E0 (exact checks) and E4 ("the standard's G" against random generators). **No security claim.** Both experiments ask whether the generator `G` that a standard fixes changes anything that index calculus can use. The plan predicts no change: the abscissa base never reads `G`, and the relation stream depends on `G` only through a change of coordinates (PLAN §1B).

E4 PROTOCOL (frozen before any E4 run)

Frozen in the commit that adds this section. The commit hash is quoted in the results. Nothing below changes after the first E4 record is written. Any deviation is listed under "Deviations" in the results.

Population. Eight 24-bit prime-order curves $y^2 = x^3 - 3x + b$, c1–c8, the first eight printed by `examples/a_minus_3_prime_order_curves.rs 24 8 20261019`. c1 and c2 are the dry run's curves. Parameters are in `e4/generators.jsonl`.

Cells (generators) and strata (curves). Five cells per curve:

- `gstd`: the finder's point with the smallest abscissa, standing in for "the standard's G";
- `grand1`–`grand4`: `[c]gstd`, with `c` drawn by `random.Random(20261019 + i)` in `[2, r − 2]` (`e4/make_specs.py`). On a prime-order curve this is a uniform non-identity point.

A/A cell. `gstd` runs again in its own session with `target_seed` 20261020: same generator, independent targets, independent algorithm seeds.

Arms. Both run on every cell with `B = ceil(r^(1/3))` and `mitm:negation_folded=1,m=2`, default (walk) targets:

- `ic-abscissa`: `prime-abscissa:size=B`;
- `ic-knownlog-random`: `known-log:size=B,scalars=random`.

Size. 2000 `planted_shared` targets per cell, one measured round, no warm-up. That makes 8 × 5 × 2000 × 2 = 160 000 main records and 8 × 2000 × 2 = 32 000 A/A records. The binding metric is the known-log arm's yield. It finds one relation per run, so a cell's yield rests on about 2000 relations. `gstd` pools 16 000 runs over the curves and `grand` 64 000. With a geometric trial count (CV ≈ 1), the gstd/grand ratio has SE ≈ √(1/16 000 + 1/64 000) = 0.88%, so the 90% half-width is about ±1.45%. That is inside ±2%. A planted +3% shift is then 3.4 SE, and the detection rule below fires with probability about 0.96. The abscissa arm finds about B relations per run, so its yield is about 14 times tighter. Total S varies by about 2.5% per run, so its interval is far inside ±2% in both arms.

Metrics, per run:

- yield `relations_found / targets_tried`, pooled as Σfound / Σtried;
- total S = (total GAE + w · unpriced ops) / √r at $w \in \{0, 0.1, 1\}$ (AGENTS.md §5a), pooled as the mean.

Primary test: equivalence of gstd and grand. For each arm and metric, `R = gstd / grand` is the equal-weight geometric mean over the eight curves of the per-curve ratio. In each ratio, `grand` pools the four random cells. The 90% interval comes from 10 000 bootstrap resamples of runs within each (curve, cell), percentile method, seed 20261019.

- **Equivalent:** the 90% interval lies inside [0.98, 1.02].
- **Different:** the interval excludes 1 and is not inside [0.98, 1.02].
- **Inconclusive:** anything else.

Secondary tests: heterogeneity among generators. For each arm and metric:

- the Cochran Q of cell means about their curve means (df = 8 × 4 = 32);
- its DerSimonian–Laird τ;
- the largest standardised cell deviation max|z|.

Each statistic gets a permutation null: runs are shuffled among the five cells of each curve, 2000 permutations. We report the p of Q and of max|z|, and τ beside the null's 95th percentile. These tests are descriptive. A $p < 0.05$ is reported as heterogeneity to investigate. It does not override the primary reading, and with 2 arms × 4 metrics, about 0.4 such flags are expected by chance.

Controls (run on the same records).

- **A/A:** `gstd` against `gstd-aa`, the same ratio and interval. It must read "equivalent" on every arm and metric. Q, τ and max|z| are reported over the two-cell partition.
- **Planted +3%:** the analysis multiplies the gstd cell's yield numerator (Σfound) or S by 1.03 on every curve and recomputes `R`. The plant is **detected** when the interval's lower end is above 1. If the plant goes undetected for an arm and metric, E4 is **inconclusive** for that arm and metric, whatever the unplanted reading.

Exact by-products (checked, not tested). By E0a the abscissa base, and so its pair table, is the same for every generator. The known-log base uses the same scalars under every `G`, so its construction cost and its pair-table count do not depend on `G`. Base construction and oracle set-up GAE must be identical across a curve's five cells, record for record. Any difference is a bug.

Prediction. Equivalent on every arm and metric. A/A equivalent. Plant detected.

Tooling. `examples/generator_effect_e4_analysis.rs` computes every number; Python only draws `e4/analysis.json`. Records are ecbench sessions, one per spec in `e4/specs/`. The run tree lands as `e4/runs.tar.xz` (AGENTS.md §13a). Host: L0, shared and loaded. Operation counts and GAE are exact and deterministic. Wall time is reported as run time only.

Results

Run on 2026-10-19. Protocol frozen in commit `b3eba557b` before the first E4 record. Host L0: shared, about 46 load on 14 cores, no pinning. Every number below is an exact operation count or derived from one. The analysis is `examples/generator_effect_e4_analysis.rs` ⟶ `e4/analysis.json`, and the figure is `e4/plot_e4.py` ⟶ `e4/e4.svg`.

Summary. E0 holds exactly: the generator changes nothing but the scalar multiplications that use it. E4 reads **equivalent within ±2% (90% CI) on every arm and metric**, and the planted +3% shift is detected on every arm and metric. One control misses its pre-registered condition. The known-log yield A/A is inconclusive, because the protocol sized the A/A comparison too small; see Deviations. Whether `G` is the curve's smallest-abscissa point or a random point changes neither the abscissa base nor the random-scalar known-log base by any measurable amount on these curves. No security statement follows; this is a generic-precomputation setting and no exponent moves.

E0: the generator carries no information (exact)

Run as cargo tests in `src/cryptanalysis/ic_boundary.rs` (`generator_coupling_tests`). The output is in `e0/e0_sweep.log`: `cargo test --release --lib generator_coupling -- --include-ignored --nocapture`, 40 s.

The hook. `CoeffScale` (a new `pub struct`, used through `collect_and_solve_relations_scaled`) replaces every drawn coefficient `a` by $a \cdot c^{-1} \bmod r$ when the run's generator is `[c]g0`. That covers random targets, walk starts, the 16 jumps, lazy and eager pool offsets, prime-walk lanes, the stride, and the stride's renewal step. It consumes the random stream exactly as the unhooked loop does. `collect_and_solve_relations` is now a wrapper that passes `None`, so every existing caller is unchanged.

Curves. c1–c6 of `e4/generators.jsonl`, B = ceil(r^(1/3)) = 205, 241, 254, 221, 252 and 213. Generators: `g0` (smallest abscissa), `[c]g0` for three random `c`, and `−g0`.

check	scope	result
E0a: digest of `prime-abscissa:size=B` (points, negation map, columns, coefficients, abscissae), built directly and through the framework builder with the generator in instance and context	6 curves × 5 generators	**30/30 identical**, one digest per curve
E0b: same target-point stream (`b` equal, $a = a_0 \cdot c^{-1}$); same relation rows and row statuses, right-hand side $h \cdot a$ scaled by c^{-1}; every pinned log = $\log_0 \cdot c^{-1}$; $k' = k \cdot c^{-1}$ recovered and verified; every native counter, outcome count and linear-algebra ledger equal	6 curves × 6 target sources × 4 generators × 2 completion rules	**288/288 exact** (+12 in the cheap default test)
E0b, ΔS: $\texttt{run} + \Sigma[a_0]g0 = \texttt{base} + \Sigma[a_0 c^{-1}]gc$ over every recorded draw, group-op ledger field by field, and the same for the verification's `[d]G`	as above	**exact in all 300**; measured ΔGAE equals the recomputed scalar-multiplication ΔGAE
E0c: target sources	`random`, `walk`, `stride`, `stride-renew`, plus `walk-prime-fast` and `walk-prime-lanes(4, batched)`	all six covered by the rows above

Under the full-rank rule every factor-base log is pinned: B + 1 = 206–255 columns, all equal to $\texttt{logs}_0 \cdot c^{-1}$ entry by entry. Under the historical target-pinned rule, 1–233 columns are pinned, and those match too. The measured ΔGAE ranges from −1884 to +2140 for `random` targets (two scalar multiplications per target) and from −106 to +47 for walks (only the jumps and offsets are scalar multiplications). It is never anything but the recomputed difference. The one native counter that moves, `walk_pool_ops`, is the pool offsets' own group ledger, scalar multiplications included, so it falls under the ΔS check. Nothing in E0 deviated.

E4: the standard's G against random generators

What ran. 16 ecbench sessions from `e4/specs/` (`e4/run_e4.sh`): 8 main sessions of 20 000 records and 8 A/A sessions of 4 000, so **192 000 records, all verified**. `ecbench verify --replay 10` on every session gives "audit OK, 10 replays reproduced, 0 problems" (`e4/verify.txt`). **Run time:** 48 min wall from 22:15:49Z to 23:03:54Z, four sessions at a time, and 9 769 s summed over sessions (875–1 164 s per main session, 162–365 s per A/A session) on the loaded host.

curve	session (main)	session (A/A)
c1	ECBS1h5482084cbe02	ECBS1hba211a0e071e
c2	ECBS1h172c8ce5a162	ECBS1h47e14b4b3342
c3	ECBS1hc2d8241ad1ba	ECBS1h70538d39d667
c4	ECBS1h2db857bc6761	ECBS1hf2f98b50150c
c5	ECBS1h48a0812f6528	ECBS1h70422a44cae0
c6	ECBS1he3de0a146b14	ECBS1h6a8a38bf38a3
c7	ECBS1h412a8daba919	ECBS1h261d45f24f5c
c8	ECBS1h3f5a2fdfff6d	ECBS1hbf40278ea7dc

Primary: gstd / grand (geometric mean over 8 curves, 90% bootstrap CI, verdict against [0.98, 1.02]):

arm	yield	S, w = 0	S, w = 0.1	S, w = 1
abscissa	0.9994 [0.9980, 1.0007] **equiv.**	1.0001 [0.9993, 1.0008] **equiv.**	1.0001 [0.9992, 1.0009] **equiv.**	1.0002 [0.9986, 1.0017] **equiv.**
known-log, random	0.9988 [0.9846, 1.0135] **equiv.**	1.0000 [1.0000, 1.0000] **equiv.**	1.0000 [1.0000, 1.0000] **equiv.**	1.0000 [0.9999, 1.0001] **equiv.**

The known-log yield interval, ±1.45%, is the width the protocol's sizing predicted. Its S interval is far narrower, because 97–98% of that arm's S is base construction and pair table, which are identical in every cell (below).

Controls.

arm · metric	A/A gstd / gstd-aa	planted +3% (detected if lower end > 1)
abscissa · yield	1.0008 [0.9991, 1.0024] equiv.	1.0294 [1.0280, 1.0307] detected
abscissa · S w = 0 / 0.1 / 1	0.9998 / 0.9997 / 0.9995, all inside ±0.25%, equiv.	lower ends 1.0293 / 1.0292 / 1.0285, detected
known-log · yield	1.0044 [0.9864, 1.0226] **inconclusive**	1.0287 [1.0142, 1.0439] detected
known-log · S w = 0 / 0.1 / 1	1.0000 [1.0000, 1.0000] (w = 1: [0.9999, 1.0001]), equiv.	lower ends 1.0300 / 1.0300 / 1.0299, detected

Secondary: heterogeneity among the five generator cells (Q on df 32, 2 000 permutations):

arm · metric	Q (perm p)	τ (null 95%)	max|z| (perm p)	with the +3% plant: Q p, max|z| p
abscissa · yield	35.8 (0.30)	0.0006 (0.0012)	3.09 (**0.021**)	< 0.001, < 0.001
abscissa · S w = 0 / 0.1 / 1	35.1 / 35.0 / 34.5 (0.33 / 0.34 / 0.35)	≤ 0.0006 (≤ 0.0015)	2.34–2.37 (0.26–0.31)	< 0.001, < 0.001
known-log · yield	28.9 (0.62)	0 (0.0133)	1.82 (0.80)	0.19, 0.50
known-log · S w = 0 / 0.1 / 1	27.8–28.3 (0.67–0.69)	0 (≤ 0.0001)	1.83–1.84 (0.78–0.79)	< 0.001, < 0.001

- **One flag of eight.** The abscissa yield's max|z| p = 0.021 comes from c4 / grand1, +0.64% against its curve's mean (figure, panel C). It is not gstd: on every curve the gstd cell is inside the abscissa cells' spread. E0b shows the relation stream's law is exactly invariant under a change of generator. With 8 such tests, 0.4 flags are expected by chance, and the Bonferroni p is 0.17. Read as chance.
- **Q does not see a single-cell +3% plant on the known-log yield.** Per-cell noise there is about ±2.2% (1 SE), so Q p = 0.19. That is why the protocol makes the pooled ratio primary.

Exact by-products. On every curve and in both arms, the factor-base phase and the oracle-set-up phase (GAE and native counters) have **one distinct value across all 12 000 records** of the five generator cells and the A/A cell. For the abscissa base this is E0a. For the known-log base it is the same scalars under every `G`: base construction of B scalar multiplications and a pair table of the same count.

Descriptive pooled values (all 8 curves pooled; curves differ in `r`, so these are context only): abscissa yield 8.648e-3 (gstd, 1 944 918 relations / 224 893 501 targets) against 8.654e-3 (grand), mean S = 19.79 / 20.66 / 28.44 at w = 0 / 0.1 / 1 in both cells. Known-log yield 8.656e-3 (gstd) against 8.669e-3 (grand), mean S = 17.76 / 17.77 / 17.83 in both cells.

Deviations

1. **The known-log yield A/A control is inconclusive, not equivalent.** The protocol required the A/A to read "equivalent" on every arm and metric. It reads 1.0044 [0.9864, 1.0226]: the interval contains 1, but its upper end is past 1.02. The cause is in the protocol's sizing. It sized the gstd/grand comparison (16 000 against 64 000 runs, ±1.45%) and gave the A/A comparison the same per-curve size, 16 000 against 16 000. That gives ±1.84% (derived: $1.645 \cdot \sqrt{(2/16\,000)}$). An equivalence call at ±2% then needs a point estimate within ±0.16%, which happens in about 1 run in 9 even with no effect at all. The A/A estimate itself is 0.4 SE from 1, so it gives no sign that the interval machinery is biased. The abscissa yield A/A and all six S A/As pass. I report the known-log yield primary reading as "equivalent" with this control flagged. A follow-up A/A at 8 000 targets per curve would bring it to about ±0.9%; it was not run here, so as not to add a post-hoc arm to a frozen run.
2. **B on c2 is 241**, as `ceil(r^(1/3))` gives ($240^3 < r = 13\,902\,853 < 241^3$). The dry run used 240 on c2 while stating the same formula.
3. `examples/generator_effect_e4_analysis.rs` had one clippy fix after the freeze (`repeat(c).take(n)` → `repeat_n(c, n)`), which leaves what it computes unchanged.
4. Conductor: the task server returned 404 from this session, so no task was claimed.

What this answers

PLAN §1B predicted, and E0 now checks exactly, that a generator `[c]G` is a change of coordinates for the abscissa base's relation search. Rows are equal, logs scale by c^{-1}, and the only cost difference is the bit pattern of the scalars the generator is multiplied by. E4 measures the same thing in law for a "standard-looking" `G` (smallest abscissa) against random ones, under both bases: equivalent within ±2% at 90% on yield and on total S at every band weight. So fixing `G` gives index calculus nothing. It gives no knowledge about a factor base, and no base built from `G` that does better because it is *this* `G`. What a fixed `G` does give, amortising one base's logs across every key, is PLAN §1C and the known-log rows of the dry run, unchanged here.

Scope: 24-bit prime-order `a = −3` curves; MITM `m = 2`; walk targets in E4, and all six target sources in E0. Nothing about P-192/P-224 beyond the algebra of §1B, which these checks confirm in code. No security claim; no improvement over the IC baseline is claimed (E4 compares a variable, not a method).

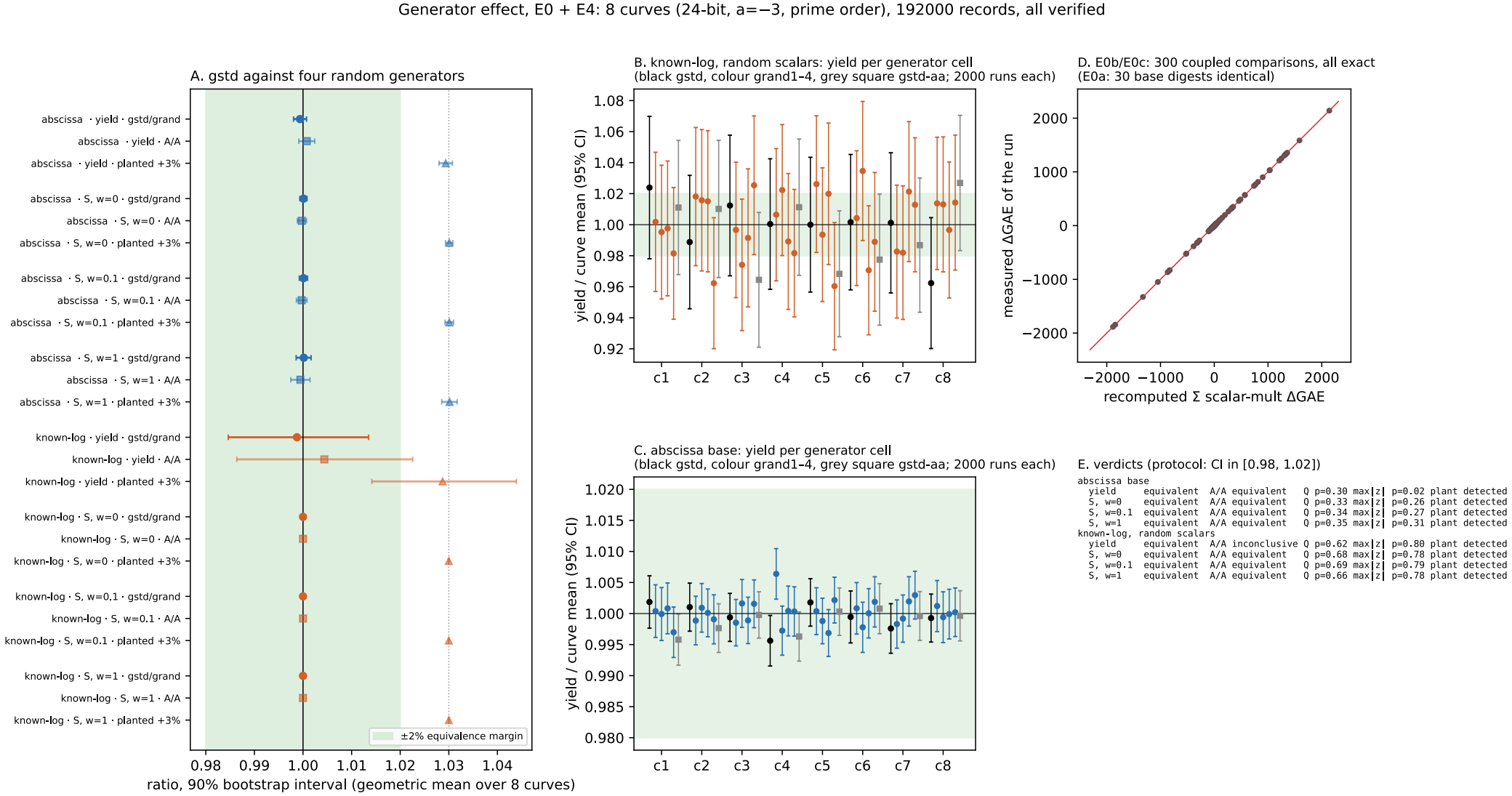


Figure 1: E0 and E4

Figure. A: gstd/grand ratio, A/A and planted +3% per arm and metric, 90% bootstrap intervals, ±2% margin shaded. B, C: yield per generator cell over its curve's five-cell mean, 95% CI, 2000 runs per cell (black gstd, colour grand1–4, grey gstd-aa). D: E0b measured ΔGAE against the recomputed scalar-multiplication ΔGAE, 300 coupled comparisons. E: verdicts. Sessions in the table above; 192 000 verified records.

Reproduce

```
cargo test --release --lib generator_coupling -- --include-ignored --nocapture   # E0
python3 research/generator_effect_ic_20261019/e4/make_specs.py                    # specs (from e4/)
research/generator_effect_ic_20261019/e4/run_e4.sh                                # E4 sessions
cargo run --release --example generator_effect_e4_analysis -- \
  e4/analysis.json e4/generators.jsonl e4/runs/e4-c[1-8] e4/runs/e4-c[1-8]-aa  # from the study dir
```

The run tree is `e4/runs.tar.xz` (`sha256sum -c e4/runs.tar.xz.sha256`; `tar -xJf e4/runs.tar.xz -C e4`), packed with bsdtar from a sorted file list with owner, mtime (2026-10-19) and mode normalised and no macOS metadata.